Simple Recommendation System

Damir S. Danial N. Aruana Zh. Sara Y.
SE-2525

- ITP 2 Final Project
- Case 7:
  Recomendation system
- Recommends movies based on preferences
- Input data comes from:
  users.json and items.json

Problem Description

Recommending items that user has not liked yet but may like.

Output

The output must include recommendations and explanations for that choise

Preference

Users like different movies

Search

The system should find similar users with similar taste to help suggest new movies

Idea behind our project

Which idea did we have in mind while designing our project

Storing

Store users and movies in JSON files

Common likes

Use common liked movies as similarity

Example

user 1 liked movie1 movie2
user 2 liked movie2 movie3
same movie movie2
recommend movie3 to user 1

Comparing

Compare users by liked movies

Recommendation

Recommend movies from similar users

System Design

The structure of our system

1

data/

users.json
items.json

2

models/

user.py
item.py
recommendation.py

3

services/

file_service.py
parser_service.py
recommendation_service.py

4

utils/

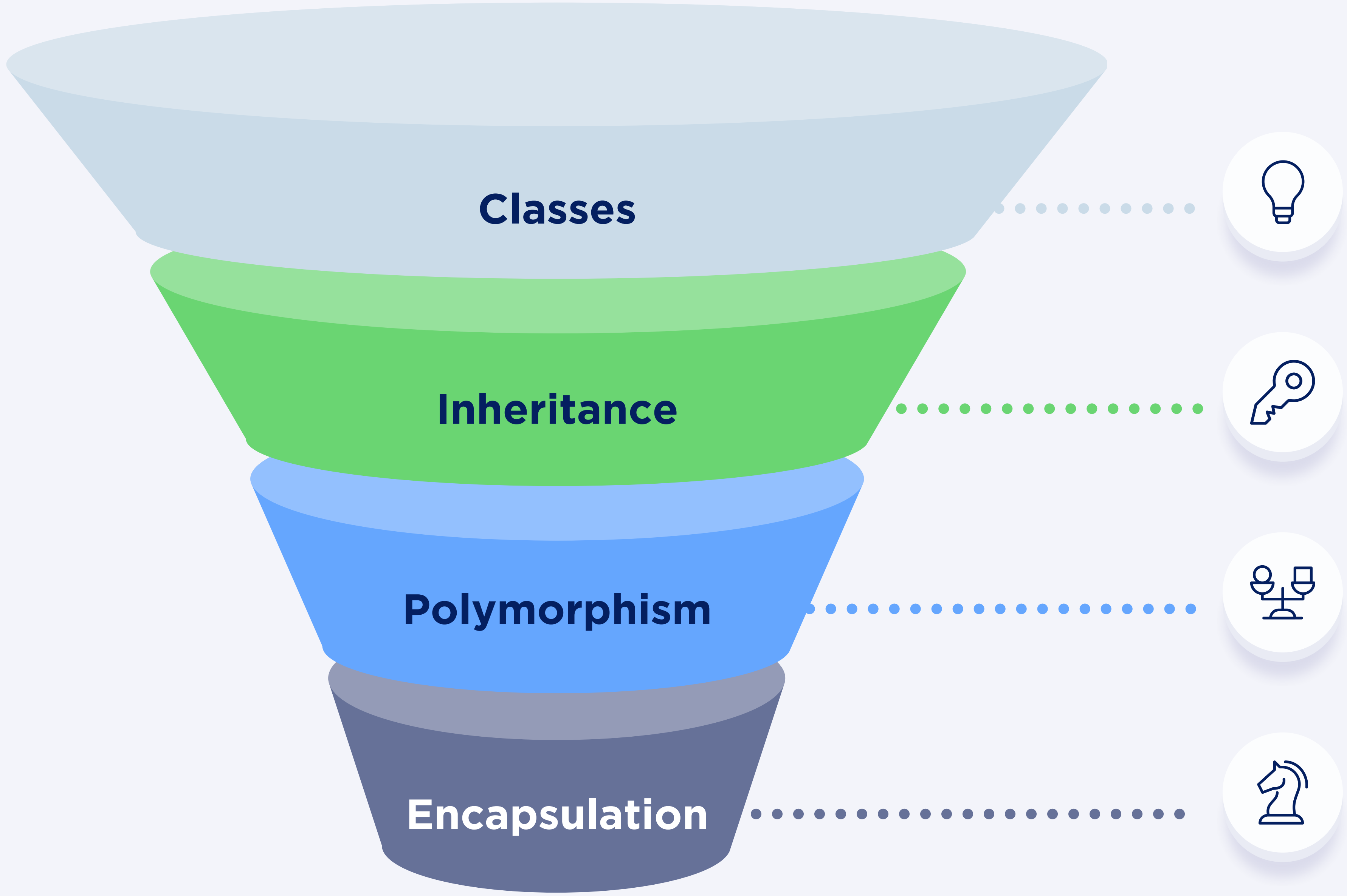
decorators.py
validators.py

5

tests/

test_recommendation.py

OOP



User, RegularUser, AdminUser, Item,
Recommendaion, RecommendationService

Regular user and AdminUser inherit from User

get_role() works differently in user classes

Fields use _field_name and properties

Data Structures

01

List

List stores users and items

02

Dictionary

Fast item lookup by id

03

Set

Finds common liked items

04

Tuple

Returns common items safely

Algorithm and features

Similarity

Similarity is calculated with set intersection

Score

Score increases when more similar users like the same item

Result

Result is sorted by score, program writes output to recommendations.json

Advanced features

Generator, decorator, lambda, regular expression

```
same_items=set1.intersection(set2)
```

Conclusion

The project solves a real recommendation problem

File handling

The project uses JSON file handling

Structure

It has modular structure

Requirements

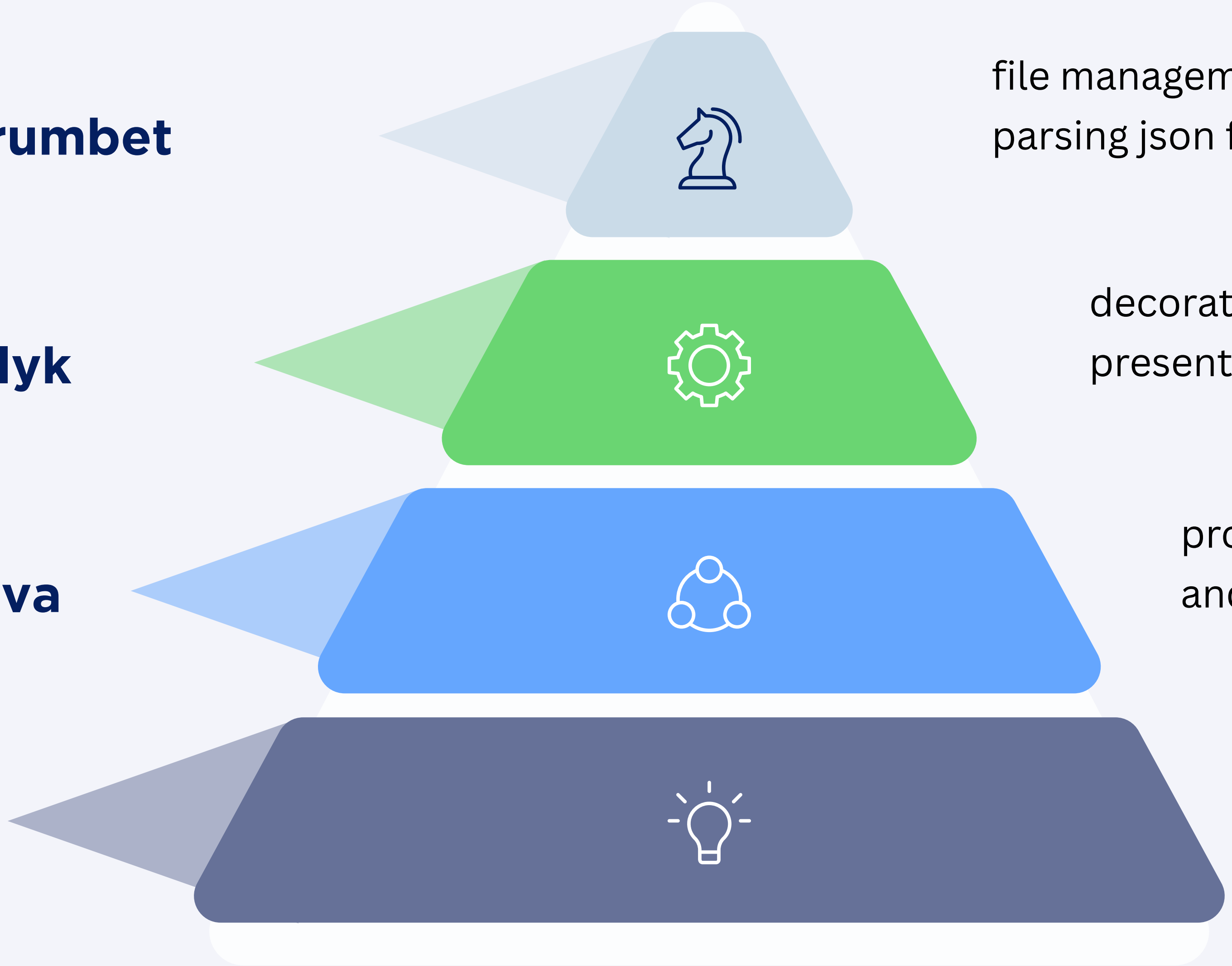
Our project includes oop, collections, testing, and algorithmic logic

Explanations

The system explains why each item was recommended

Work Distribution

How exactly did we do our project ina group of four people



Danial Nurumbet

file management
parsing json files

Aruana Zhaksylyk

decorators, file handling, validation
presentation

Sara Yermaganbetova

project description, README file, presentation, models, item and recommendation classes, user,regularUser,adminUser

Damir Satimov

Initialized project, Hierarchy, Instructions for, everyone, optimization and better logic, main, program, console outputs, tests

References:

Aggarwal, C. C. (2016). Recommender systems: The textbook. Springer. https://doi.org/10.1007/978-3-319-29659-3

Python Software Foundation. (2026). json — JSON encoder and decoder. Python documentation. https://docs.python.org/3/library/json.html

Python Software Foundation. (2026). unittest — Unit testing framework. Python documentation. https://docs.python.org/3/library/unittest.html

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). Recommender systems handbook (2nd ed.). Springer. https://doi.org/10.1007/978-1-4899-7637-6